

Esercizio

Si consideri il tipo di dato datatype `FOR = For of int * (int -> int)`; i cui valori `For(n, f)` rappresentano funzioni che implementano un ciclo for come il seguente:

```
int ciclofor(int x)
{
  for (i = 0; i < n; i++) {
    x = f(x);
  }

  return x;
}
```

Si scriva una funzione `eval` (avente tipo `FOR -> (int -> int)`) che riceve come argomento un valore di tipo `FOR` e ritorna una funzione da interi ad interi che implementa il ciclo indicato qui sopra (applica `n` volte la funzione `f` all'argomento).

Come esempio, se `val f = fn x => x * 2`, allora `eval (For(3, f))` ritornera' una funzione che dato un numero `i` ritorna `i * 8`:

```
> val f = fn x => x * 2;
val f = fn: int -> int
> eval (For(3, f));
val it = fn: int -> int
> val g = eval (For(3, f));
val g = fn: int -> int
> g 5;
val it = 40: int
```

IMPORTANTE: notare il tipo della funzione!.

La funzione `eval` deve essere definita in un file `.sml` autocontenuto ma separato da qualsiasi codice di test si sia usato. Non e' necessario che contenga la [definizione del tipo di dato FOR](#). Si consegna il file `.sml` contenente la definizione di `eval`.

Possibile Soluzione

Questa e' una [possibile soluzione](#).